# Glaciem (GLAC) — A Mac-First, CPU-Only Privacy Coin

Whitepaper v1.0

Glaciem Project

20 May 2026

**Abstract**

Glaciem (GLAC) is a privacy-focused cryptocurrency forked from Monero, with an original CPU-only proof-of-work called *Lattice* and a deliberately Mac-first toolchain. Glaciem inherits Monero's privacy model — Ring Confidential Transactions, ring signatures, Bulletproofs+, and stealth addresses — without modification, and its monetary policy (two-minute blocks, smooth emission, tail emission, no premine) almost without modification. Lattice replaces RandomX as the consensus PoW: a single-phase, integer-only walk over a large per-nonce scratchpad and a shared epoch dataset, designed for wide high-IPC CPU cores and explicitly hostile to GPU occupancy. Native miner-plus-wallet applications ship for macOS, Windows, Android, and Linux. Glaciem launched mainnet on 20 May 2026 as an explicitly experimental coin: Lattice has not yet received external cryptographic review, GLAC has no exchange listing or market price, and miners are explicitly told the coin is not an investment. This whitepaper documents the design, the strategy, the security assumptions, the open risks, and the design dead-end that preceded Lattice.

## Contents

# 1 1. Introduction

## 1.1 1.1 Motivation

Monero's RandomX proof-of-work is one of the most successful CPU-friendly mining algorithms ever deployed. It is also, after years of optimisation, dominated in absolute terms by large EPYC-class servers in hosted datacenters. A user mining on the desktop they already own competes against rented compute, and the resulting hashrate share is typically vanishingly small.

Glaciem starts from a different premise. Rather than aim for hardware-agnostic CPU mining, it leans deliberately into one corner of the consumer CPU market: modern Apple Silicon. The reasoning is economic rather than algorithmic — elaborated in §5 — and it shapes everything else:

- The proof-of-work, *Lattice*, is single-phase, integer-only, and built around primitives that are friendly to wide, high-IPC, branchy CPU cores.
- Native miner-and-wallet apps ship as polished first-party software for the four platforms most likely to be in users' hands: macOS, Windows, Android, and Linux. The barrier from "I heard about Glaciem" to "I am mining" is meant to be one download.
- Everything else — privacy, monetary policy, the wider consensus protocol — is inherited from Monero, unchanged, and credited to the Monero Project (§2). Glaciem does not re-litigate the privacy stack.

## 1.2 1.2 What Glaciem is and is not

Glaciem **is**:

- A working privacy coin: ring signatures, RingCT, Bulletproofs+, and stealth addresses are mandatory and unchanged from Monero.
- A live, mineable network as of 20 May 2026 at 00:00 EDT, with no premine and no founder reward.
- An attempt to put a serious CPU-friendly mining experience in the hands of users on hardware they already own.

Glaciem **is not**:

- A finished or audited project. Lattice is an original PoW and has not yet had external cryptographic review (§8.4, §10). This is the single largest open risk.
- An investment. GLAC has no exchange listing, no market price, no liquidity, and no commitment that any of those will exist. Miners are told plainly to mine at their own risk.
- A challenger to Monero. The privacy machinery here is Monero's, used gratefully and with attribution. Glaciem changes the proof-of-work and the brand around it; it does not claim to improve on Monero's cryptography.

The names "Glaciem" and "GLAC" are working names pending a trademark check and may change.

# 2 2. Inherited from Monero

Glaciem is forked from `monero-project/monero` at commit `67c283c98` (`v0.18.0.0-1653-g67c283c98`), dated 16 May 2026. The upstream remote is preserved as `upstream` so that future Monero fixes — particularly security fixes — can be merged into Glaciem with minimum drift.

The following components are inherited unchanged. Each one represents many person-years of work by the Monero Project and the wider CryptoNote community, to whom credit belongs:

## 2.1 2.1 Privacy model

- **Ring signatures** with a fixed ring size, hiding the true sender of each transaction among decoys.
- **Stealth addresses** (one-time public keys per output) such that no two payments to the same recipient share an on-chain address.
- **RingCT (Ring Confidential Transactions)**, hiding transaction amounts while preserving consensus-verifiable balance.
- **Bulletproofs+** range proofs, ensuring committed amounts are non-negative without revealing them.

All four are mandatory on every Glaciem transaction. Glaciem does not introduce optional or "transparent" modes.

## 2.2 2.2 Monetary policy

- **Block target**: 120 seconds (`DIFFICULTY_TARGET_V2`).
- **Atomic units**: 1 GLAC = $10^{12}$ atomic units (`COIN` = $10^{12}$).
- **Money supply**: Monero's atomic supply model is preserved (`MONEY_SUPPLY = 2^{64}-1`), with smooth emission governed by `EMISSION_SPEED_FACTOR_PER_MINUTE = 20`. The block reward decays geometrically until reaching the tail-emission floor.
- **Tail emission**: Monero's perpetual tail-emission floor is preserved so the security budget never goes to zero. The exact tail-reward value is inherited from upstream and may be re-examined in a future hard fork.

- **No premine**: the genesis block is fresh and empty. The first coinbase is at block 1 and is mined like any other.
- **No founder reward** and no per-block tax to a developer address.

### 2.3 2.3 Network identity

The fork is *network-identity* rebranded in line with the CryptoNote/Monero convention for distinct networks:

- Address prefix `144` on mainnet, decoding to a base58 prefix of `R` — every Glaciem mainnet address begins with `R`. Testnet (`528`) and stagenet (`912`) addresses likewise begin with `R` but are mutually unspendable.
- Distinct P2P network IDs and default ports (P2P `19080`, restricted RPC `19099`, RPC `19081`).
- The hardcoded Monero seed-node list, DNS checkpoints, and developer checkpoints are removed. Glaciem ships its own seed list and its own bootstrap-window checkpoint policy (§8.5).

## 3 3. Lattice — the Glaciem Proof-of-Work

This section is a condensed version of the full design notes (`POW_DESIGN.md`); the canonical algorithm is the reference C implementation at `pow/lattice_ref.c`, which the consensus copy at `src/crypto/lattice.{c,h}` is bit-identical to.

### 3.1 3.1 Design goals

Lattice is built around five requirements, in priority order:

1. **CPU-only.** No GPU phase, no GPU verification path, no GPU mining path. One kind of work, one kind of hardware.
2. **Deterministic.** Bit-identical output on every platform. Integer-only, defined-overflow arithmetic. No floating point anywhere.
3. **Verifier-cheap.** A full node validates a block in a few milliseconds of CPU work and no special hardware.
4. **Wide-CPU favouring.** A wide, high-IPC core with deep out-of-order execution mines faster than a narrow one. This is the property that biases mining toward modern Apple Silicon (§5).
5. **GPU-hostile.** Per-nonce working-set size and serial dependency structure that collapses GPU occupancy in the RandomX style.

A sixth, non-functional requirement is **review surface**: the entire algorithm should reduce to one well-analysed cryptographic primitive, so the eventual external review is bounded in scope. Lattice's sole primitive is `arx_perm()`, the BLAKE2b round function — a 64-bit ARX permutation over a 1024-bit state, widely deployed and extensively analysed.

### 3.2 3.2 The walk

One Lattice evaluation of a `(header, nonce)` pair proceeds as four steps:

1. **Seed.** The `header || nonce` blob is sponge-absorbed into a 2 KiB working state (`ST_WORDS = 256` 64-bit words).
2. **Scratchpad.** A 1 MiB per-nonce scratchpad (`SCRATCH_WORDS`) is filled with an `arx_perm` keystream derived from the seeded state.

3. **Walk.** A sequential, data-dependent loop runs for `L_WALK = 32{,}768` iterations. Each iteration:
    - Derives scratchpad, dataset, and state block indices from an accumulator.
    - Mixes the addressed blocks into the accumulator.
    - Applies `arx_perm` *once or twice* depending on a data-dependent bit (a branch — and therefore a warp-divergence on a GPU).
    - Writes the result back into both the scratchpad and the working state.
4. **Finalize.** The state is sponge-squeezed to a 32-byte hash, which is compared against the difficulty target.

Two properties make the walk hostile to parallelism within a nonce:

- **Latency-bound serial dependency.** Each iteration's inputs depend on the previous iteration's outputs. There is no instruction-level parallelism *within* a single nonce — a wider, deeper out-of-order CPU core wins.
- **A 1 MiB per-nonce live working set.** A GPU mines by running thousands of nonces in parallel, and thousands × 1 MiB exceeds any consumer GPU's on-chip memory by orders of magnitude. Per-thread occupancy collapses. This is the same mechanism RandomX uses — the proven source of GPU-farm resistance in the Monero ecosystem.

### 3.3 3.3 The epoch dataset

A 4 MiB global dataset (`DATASET_WORDS`) is filled by an `arx_perm` keystream from an *epoch seed*. The epoch seed is a recent block hash (the RandomX construction), so it is stable across an epoch: a miner builds the dataset **once per epoch**, not per nonce, and the cost amortises away. The walk reads it at data-dependent indices and never writes to it.

### 3.4 3.4 Verification

A validating node — any platform, any architecture, no GPU — verifies a block by running **one** walk evaluation for the block's nonce in integer math and checking `hash $\leq$ target`. The dataset is built once per epoch and reused. Per-block verification is a single walk: a few milliseconds, in the same ballpark as RandomX. The block carries only the nonce and standard header fields — no GPU proof, no extra artefacts, no auxiliary state.

This is what makes Lattice cheap to verify and trivially portable. Any Monero-style full node, on any hardware, validates Glaciem exactly the same way it validates any chain.

### 3.5 3.5 Determinism

Consensus on a PoW requires bit-identical results across every node on every platform. Lattice ensures this with:

- **Integer-only arithmetic.** Floating-point operations are not portable across architectures and toolchains; they are excluded.
- **Defined overflow.** Wrapping (modular) arithmetic only; no undefined-behaviour reliance.
- **Fixed sizes.** Walk length, state size, scratchpad size, dataset size — all are consensus constants and not negotiable per-node.

Because Lattice is CPU-only and integer-only, there are no GPU kernels to keep bit-identical across vendors, and no cross-architecture floating-point behaviour to police. This is a real simplification compared to Lattice's two-phase predecessor (§5), which required Metal and OpenCL kernels each cross-validated against the C reference.

### 3.6 3.6 Parameters

| Constant | Value | Meaning |
|---|---|---|
| `ARX_ROUNDS` | 8 | rounds inside `arx_perm` |
| `ST_WORDS` | 256 (2 KiB) | working state, per nonce |
| `SCRATCH_WORDS` | $2^{17}$ (1 MiB) | per-nonce scratchpad — tunable |
| `DATASET_WORDS` | $2^{19}$ (4 MiB) | global per-epoch dataset |
| `L_WALK` | 32,768 | walk iterations per nonce — tunable |

`SCRATCH_WORDS` and `L_WALK` are the two intentional tuning knobs. `SCRATCH_WORDS` is sized to fit comfortably in a large consumer CPU's L2/L3 cache while still being far too large for GPU on-chip memory at scale. `L_WALK` sets the wall-clock cost of one evaluation; the current value targets a single-walk verification in the low single-digit milliseconds. Both values are *provisional* — chosen for a working algorithm, not yet tuned by measurement (§10). A future hard fork will deliver measurement-driven values.

### 3.7 3.7 Pseudocode

The full reference is `pow/lattice_ref.c`. Compressed pseudocode for one nonce evaluation:

```
function lattice(header, nonce, dataset):
    state    = sponge_absorb(header || nonce, ST_WORDS)
    scratch  = keystream(state, SCRATCH_WORDS)        // arx_perm
    acc      = state[0]
    for i in 0 .. L_WALK - 1:
        s_idx = acc & (SCRATCH_WORDS - 1)
        d_idx = (acc >> 17) & (DATASET_WORDS - 1)
        t_idx = (acc >> 36) & (ST_WORDS    - 1)
        block = scratch[s_idx] ^ dataset[d_idx] ^ state[t_idx]
        acc   = mix(acc, block)
        if (acc & 1):
            block = arx_perm(arx_perm(block))         // 2 perms
        else:
            block = arx_perm(block)                   // 1 perm  (branch)
        scratch[s_idx] = block
        state[t_idx]  ^= block
    return sponge_squeeze(state, 32)
```

(Block sizes, exact mix functions, and index derivations are in the reference implementation. The pseudocode above is illustrative, not normative.)

## 4 4. Why CPU-only

The decision to be *CPU-only* — not merely *CPU-friendly* — is the single most consequential decision in Lattice's design. It removes the largest single mining-economics risk a consumer-grade PoW faces, namely GPU farms, and it makes the verification path universally cheap.

A CPU-only PoW is not novel. RandomX is CPU-only and dominates Monero mining because of it. Lattice differs from RandomX in two practical ways: it is single-phase (one algorithm, not virtual-machine-plus-reduction), and it uses a single cryptographic primitive (`arx_perm`) chosen so that the eventual external review has a small, well-understood surface.

What *is* novel in Glaciem is not "CPU-only" — it is the explicit acknowledgment that even CPU-only mining concentrates somewhere, and the intentional choice to lean into modern Apple Silicon as the target concentration. The reasoning is economic, not algorithmic, and is the subject of §5.

# 5 5. The Apple-Silicon Strategy

## 5.1 5.1 What does not work: the structural moat

An early version of Glaciem aimed for something stronger than a market tilt: a *structural moat* that would make Apple Silicon out-mine other hardware by virtue of the algorithm itself. The mechanism was a two-phase CPU + GPU pipeline in which a per-stage hand-off was free on Apple's unified-memory architecture and would be expensive on a discrete GPU crossing PCIe. The intent was that the hand-off tax would shift the hardware optimum toward unified-memory machines.

That claim was tested empirically and is now retired. The reasoning is recorded here on purpose, both because the lesson is important and because the failure mode is *structural*, not implementation-specific.

A consensus PoW is, by construction, a **portable deterministic function any node can verify cheaply**. Those three properties form a cage:

- The verifier and the miner compute *the same function*. Whatever path is cheap for the verifier is available to the miner. An algorithm cannot hide "expensive on hardware X, cheap on hardware Y" — both verifier and miner have to compute it.
- The function can be biased toward a **resource** (memory capacity, latency, branch density, cache footprint) but never toward a **vendor**. For every resource Apple is good at, some non-Apple machine is better: memory channels → multi-socket servers; matrix throughput → datacenter GPUs; raw bandwidth → HBM.
- The **miner**, not the designer, chooses how to map the function onto silicon. The designer cannot force a CPU↔GPU split or a cross-socket hop — a miner with a different machine will simply skip it.

## 5.2 5.2 The empirical record

Profiling on a real discrete GPU — an NVIDIA Tesla T4 — with OpenCL event profiling measuring time in CPU↔GPU transfers versus GPU compute:

- A ~128 KB per-stage hand-off: transfer time was **0.0%** of mining time.
- Widening the hand-off **32×** to ~4 MiB: transfer time was still **0.2%**.

The ratio cannot improve, because transfer time and compute time both scale linearly with state size — the ratio is size-independent and compute dominates. A hand-off can never be the bottleneck. Worse, a hand-off heavy enough to matter would also make the §3.4 CPU-only verification expensive. The two-phase architecture, the Metal and OpenCL kernels, and the profiling miner were all retired once this was settled. Lattice is what remained.

## 5.3 5.3 What replaces it: the economic gravity well

Lattice's advantage for Apple Silicon is **economic, not structural** — and in crypto, that is the correct goal. Miners deploy whatever hardware is most profitable; profitability is *revenue minus electricity*, amortised over hardware cost. Two economic facts favour Apple Silicon, and Lattice is shaped to lean on them:

1. **Performance per watt.** Apple's wide, deeply out-of-order CPU cores lead the consumer field on compute-per-watt. A CPU-hard, latency-bound, branchy algorithm puts the contest squarely on CPUs — the terrain where that lead is largest and least contested. Where electricity is normally priced, an Apple machine is simply the most profitable consumer box to mine on. Lattice being *CPU-only* — no GPU phase at all — sharpens this: the GPU is the one place Apple's perf-per-watt lead narrows, and Lattice never goes there.
2. **Barrier to entry.** Tens of millions of Macs already exist, owned and idle. A near-zero-friction native miner — generate a wallet, tap mine — puts the optimal hardware in the user's hands on day one. This is the part of the advantage fully under Glaciem's control and the part that cannot be competed away.

The outcome is organic. No consensus rule forces Macs to win. The market does, because Macs are the most profitable consumer box for this workload — most strongly where electricity is priced at consumer-grid rates.

### 5.4 5.4 Honest limits

- The economic well is **strong where electricity is expensive, weak where it is cheap**. A miner on subsidised industrial power optimises hardware cost and absolute hashrate, not perf-per-watt. The Apple tilt is regional, not universal.
- A perf-per-watt edge **arbitrages into farms** of the favoured device. Mac mining farms are still Mac mining, but they are not the casual users the design is aimed at.
- The durable, un-competable piece is the **low barrier to entry** — wide pre-existing ownership of capable hardware combined with first-party native software. That is what the project actually controls and invests in.

## 6 6. Network Architecture

Glaciem's network is intentionally close to Monero's. The interesting deviations are at the *client* layer, not the protocol layer.

### 6.1 6.1 Node topology

Full nodes run `rimed`, the Glaciem fork of `monerod`. Defaults:

- P2P port `19080`
- Unrestricted RPC `19081`
- Restricted RPC `19099`

P2P, block propagation, mempool, peer-exchange, and the wire protocol are unmodified from upstream Monero. Anyone who can run a Monero full node can run a Glaciem full node, with the same operational practices.

At launch the project operates two bootstrap nodes on commodity VPS infrastructure. The expectation, and the explicit goal, is that the volunteer-run node pool grows beyond the bootstrap operators over time. The planned *masternodes* roadmap item (§8.3) is the deliberate economic incentive for that growth.

### 6.2 6.2 The client-side proxy layer

The four official miner apps (macOS, Windows, Android, Linux) do not by default talk to a specific node IP. They route their JSON-RPC and binary RPC calls through a Cloudflare

Worker reverse proxy. The Worker fails over between origin nodes; if the first origin returns a 5xx or transport error the request retries against the next.

This layer exists for two reasons:

1. **Absorbing connection storms.** A consumer miner finding a low-difficulty block in a tight loop produces a connection pattern that resembles a small DDoS. Cloudflare absorbs it before it reaches the origin nodes.
2. **Failover for users.** When a single origin node is unreachable — whether due to a Hetzner DDoS scrub, a node restart, or a kernel upgrade — the Worker quietly retries on another. End users do not see the per-node outage.

The Worker is a single integration point between the apps and the network. That is a feature for typical operation and a *single point of failure* for the Worker itself. The user-side mitigation is to keep the per-node host and port configurable in every miner app, so a power user can point at any reachable node directly. A planned future improvement is *client-side fallback*: shipping the apps with a list of seed nodes that they fall back to automatically when the Worker is unreachable.

### 6.3 6.3 Embedded miners and wallets

The four miners are first-party native applications. Each one bundles the upstream Monero `wallet_api` — i.e. the same C++ wallet library that powers `monero-wallet-cli` — behind a C ABI (`pow/wallet/rime_wallet.h`). The result is that every official miner is also a full wallet: generate an address, sync, send, receive, sweep, and view transaction history without running a separate `wallet-rpc` daemon.

- **macOS**: SwiftUI, native Cocoa, ships as `Glaciem Miner.app`.
- **Windows**: Win32 + WinHTTP, ships as a single self-contained `.exe`.
- **Android**: Jetpack Compose, ships as an APK with a `MiningMode` selector (Eco / Balanced / Max) for thermals and battery.
- **Linux**: Qt6/QML, ships as an `.AppImage` for any modern x86_64 distro.

A fifth client path — a CLI build-from-source for distros where the AppImage is not desired — is documented in `BUILD_LINUX.md`.

## 7 7. Monetary Policy

Glaciem's monetary policy is Monero's monetary policy, with no current modifications. The salient values:

| Property | Value |
|---|---|
| Block target | 120 seconds |
| Atomic units | $10^{12}$ per GLAC (12 decimal places) |
| Initial reward | inherited from Monero's emission curve at block 1 |
| Reward decay | smooth, `EMISSION_SPEED_FACTOR_PER_MINUTE = 20` |
| Tail emission | preserved from Monero — perpetual non-zero reward |
| Premine | none |
| Founder reward | none |
| Per-block tax | none |
| Coinbase maturity | 60 blocks (inherited from Monero) |

A few points worth elaborating:

- **Fair launch.** The mainnet genesis block is empty. Block 1 is the first block with a coinbase, mined publicly. There is no founder allocation, no presale, no developer drop. Every GLAC in circulation has been mined through Lattice.
- **Tail emission.** Monero's perpetual tail emission is preserved so the security budget never decays to zero. Glaciem may revisit the tail value in a future hard fork once miner economics on the live network are better understood, but the *existence* of a tail is intentional.
- **Coinbase maturity and unmixable outputs.** Inherited mechanics: a coinbase output is locked for 60 blocks and is also unmixable until swept into a regular output. Every official wallet exposes a *Sweep Unmixable* action that does this in one click.

# 8 8. Security and Threat Model

## 8.1 8.1 GPU mining farms

Resisted. The 1 MiB per-nonce scratchpad collapses GPU occupancy by the same mechanism RandomX uses. Lattice is also CPU-only — there is no native GPU path to begin with. This is the threat Lattice is best at, and is the reason the design exists in this form.

The honest framing is *GPU-hard*, like RandomX — not *GPU-proof*. A sufficiently determined GPU implementation will exist and will hashrate; it will simply be uneconomic compared to a CPU per dollar and per watt.

## 8.2 8.2 Large multi-core CPU servers

**Not specifically defended.** Lattice is CPU-hard, and a large EPYC-class many-core server mines it well. This is the same property RandomX has and is not a defect — the economic tilt of §5.3, not the algorithm, is what favours consumer Apple Silicon over hosted compute. A datacenter server running on cheap industrial power may well out-mine an M-series Mac in absolute terms; the design is honest about that.

## 8.3 8.3 Rented cloud hashrate

Cloud GPU instances are CPU-light and GPU-heavy; a CPU-bound PoW mines poorly on them. Cloud CPU instances are available but are typically priced at higher per-watt cost than dedicated hardware. The effect is real but incidental — not a designed defence.

## 8.4 8.4 Novel-algorithm risk — *the dominant risk*

Lattice is unreviewed. A novel proof-of-work can hide one of two catastrophic shortcuts:

- An **algorithmic optimisation** that lets a knowledgeable miner compute hashes much faster than the reference implementation suggests, breaking the assumed work-per-hash and concentrating mining power among the few who know the trick.
- A **purpose-built ASIC** that maps the algorithm efficiently to custom silicon, breaking the CPU-friendliness assumption entirely.

External cryptographic review is the only thing that meaningfully reduces this risk. Mainnet launched without it. This is documented in §10 and §11 and in every public surface of the project (README, website, in-app disclaimers). Users are explicitly told the coin is experimental and not an investment.

### 8.5 8.5 Bootstrap-window 51%

A new chain with low hashrate is acutely vulnerable to a 51% attack by any actor who can briefly point enough hashrate at it. The mitigations are operational:

- **Developer checkpoints** during the bootstrap window, retired once honest hashrate is healthy. Checkpoints are a temporary trust assumption on the project team that the project is explicit about.
- **Coinbase maturity** is preserved at 60 blocks, providing a meaningful reorg window before mined coins are spendable.
- The two bootstrap nodes are peered with each other and run with conservative timeouts.

## 9 9. Roadmap

Status as of mainnet launch (20 May 2026):

| Done | Pending |
|---|---|
| [x] Lattice C reference, test vectors, self-tests | [ ] External cryptographic review of Lattice |
| [x] Consensus copy in the daemon at `HF_VERSION_LATTICE` (v18) | [ ] Measurement-driven tuning of `SCRATCH_WORDS` and `L_WALK` |
| [x] Native miner+wallet apps: macOS, Windows, Android, Linux | [ ] Masternodes — incentive layer for node operators |
| [x] Two-phase CPU+GPU predecessor disproven and retired | [ ] Mining pool support (centralised first, then p2pool-style) |
| [x] Mainnet launch — fair start, no premine | [ ] Client-side fallback node list (Worker resilience) |

External cryptographic review is the most important open item. The other items are improvements; this one is a category change — until it happens, the project should be treated as the experimental work it is.

## 10 10. Honest Limitations

The most important risks and limits, in one place:

- **Lattice is unreviewed.** §8.4 in detail. This is the project's most significant risk and is not yet mitigated.
- **Parameters are provisional.** §3.6. `SCRATCH_WORDS` and `L_WALK` were chosen for a working algorithm, not by measurement on real hardware. Tuning is a planned future hard fork.
- **The Apple-Silicon tilt is economic, not structural.** §5.4. The algorithm does not guarantee Apple wins; market conditions do, and only in regions with consumer-grid electricity prices.
- **"Glaciem" and "GLAC" are working names**, pending a trademark check. Both may change.
- **Bootstrap-window checkpoints** are a temporary trust assumption on the project team during the low-hashrate period.
- **The Cloudflare Worker is a single point of failure** in current client routing. Each app has a configurable node host as a manual workaround; client-side automatic fallback is a planned improvement.

## 11 11. References


- The Monero Project. *Monero Whitepaper* and *zero-to-Monero*. https://www.getmonero.org/library/
- Nicolas van Saberhagen. *CryptoNote v2.0.* 2013.
- tevador. *RandomX Design Document.* Monero Project. https://github.com/tevador/RandomX
- B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell. *Bulletproofs: Short Proofs for Confidential Transactions and More.* IEEE S&P 2018.
- J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, C. Winnerlein. *BLAKE2: simpler, smaller, fast as MD5.* 2013. (The `arx_perm` primitive that Lattice uses is the BLAKE2b round function.)


## 12 Appendix A — Current Consensus Constants

These are the values active on Glaciem mainnet at the time of writing (extracted from `src/cryptonote_config.h`). They are inherited from Monero unless otherwise noted.

| Constant | Value | Notes |
|---|---|---|
| `DIFFICULTY_TARGET_V2` | 120 s | block target |
| `COIN` | $10^{12}$ | atomic units per GLAC |
| `MONEY_SUPPLY` | $2^{64} - 1$ | atomic supply ceiling |
| `EMISSION_SPEED_FACTOR_PER_MINUTE` | 20 | smooth-emission decay constant |
| `CRYPTONOTE_REWARD_BLOCKS_WINDOW` | 100 | reward-window size |
| `CRYPTONOTE_BLOCK_GRANTED_FULL_REWARD_ZONE_V5` | 300000 | full-reward zone (bytes) |
| `CRYPTONOTE_PUBLIC_ADDRESS_BASE58_PREFIX` (mainnet) | 114 | base58 prefix → addresses start with `R` |
| `HF_VERSION_LATTICE` | 18 | hard-fork version Lattice activates at |

Lattice's own constants are in §3.6.

## 13 Appendix B — RPC and Network Reference

| Service | Default port | Notes |
|---|---|---|
| `rimed` P2P | 19080 | unmodified Monero peer protocol |
| `rimed` RPC | 19081 | unrestricted JSON-RPC + binary RPC |
| `rimed` restricted RPC | 19099 | public-facing restricted RPC |
| `rime-wallet-rpc` | 29083 | optional wallet RPC daemon |

Public client-routing endpoint:

- `https://glaciem-rpc.frostmine.workers.dev/json_rpc` — Cloudflare Worker in front of the bootstrap nodes; binary RPC paths pass through to the same origin pool.

Binary names retain the `rime-` prefix as a holdover from the project's pre-rename name; they will be renamed in a future build.

---

*This document is version 1.0, dated 20 May 2026. It accompanies the project's source at https://github.com/hughson/Glaciem and is licensed under the same BSD 3-Clause License as the rest of the project. The Glaciem Project gratefully acknowledges the work of the Monero Project, on which Glaciem's privacy and consensus technology is built.*